

Sustainable Smart City Assistant Using IBM Granite LLM

Project Documentation

Sustainable Smart City



Team members:

- Team leader : Mohammed Kaja Maidheen I
- Team member : Kishore Kumar V
- Team member : Udhaya kumar M
- Team member : Paveen A

S.NO	INDEX	PG.NO
1	Introduction	2
2	System Architecture	2
3	Functionalities	3
4	Code Implementation	4
5	User Interface Design	8
6	Future Enhancements	9
7	Conclusion	9

1. Introduction

Project Overview:

The Sustainable Smart City Assistant project is designed to provide eco-friendly tips, summarize policy documents, and suggest smart city innovations. The system utilizes an advanced language model, Granite-3.2, to generate responses based on user input.

Objectives:

- To offer actionable eco-friendly tips for individuals and businesses.
- To summarize policy documents, highlighting key points and implications.
- To generate innovative ideas for smart city solutions, making them practical and sustainable.

Technology Stack:

- Backend: Python, Gradio, Transformers (Hugging Face), PyTorch
- Model: IBM Granite-3.2
- Frontend: Gradio for interactive user interface

2. System Architecture

Overview:

The system is based on a client-server architecture, where the backend processes the user inputs and interacts with the Granite-3.2 language model to generate responses. The frontend, developed using Gradio, provides an interactive interface for users to input keywords and view results

Components:

Model Setup:

- The backend uses the Granite-3.2 model loaded with the AutoTokenizer and AutoModelForCausalLM from the Hugging Face Transformers library.

Backend:

- The backend performs text processing, response generation, and integrates with the Gradio interface.

Frontend:

- Gradio is used to build the interactive UI that handles user inputs and displays the generated results (eco tips, policy summaries, and smart city innovations).

Data Flow:

1. The user inputs a keyword or text in the Gradio interface.
2. The input is sent to the backend, where it is processed using the language model.
3. The model generates a response, which is sent back to the Gradio interface for display.

3. Functionality

Eco Tips Generator:

Input: User enters a keyword related to sustainability (e.g., "solar", "plastic", "water waste").

Processing: The system constructs a prompt to request eco-friendly tips related to the input keyword.

Output: The model generates 5 short and actionable eco-friendly tips.

User Interface: Tips are displayed with attractive styling (using gradients and custom fonts) in a column layout.

Policy Summarizer:

Input: User provides a policy document text.

Processing: The system generates a summary of the policy document, highlighting key provisions and implications.

Output: A summarized version of the policy, formatted for clarity.

User Interface: The summary is presented in a visually appealing way, with distinct background colors for each point.

Smart City Innovations:

Input: User enters a topic related to smart cities (e.g., "transport", "energy", "housing").

Processing: The system generates 5 innovative and futuristic smart city ideas related to the input topic.

Output: Practical and sustainable smart city ideas are displayed with attractive visual elements.

User Interface: The ideas are presented in a structured layout, styled with gradients and icons.



4. Code Implementation

Model Setup: The language model Granite-3.2 is loaded using the AutoTokenizer and AutoModelForCausalLM classes from the Hugging Face Transformers library. The model is configured to use 16-bit precision on compatible devices (if CUDA is available).

```
model_name = "ibm-granite/  
granite-3.2-2b-instruct"  
tokenizer =  
AutoTokenizer.from_pretrained(model_name)  
model =  
AutoModelForCausalLM.from_pretrained(model_name,  
  
torch_dtype=torch.float16 if  
torch.cuda.is_available() else  
torch.float32,  
  
device_map="auto" if  
torch.cuda.is_available() else None)
```

Response Generation: A `generate_response()` function is implemented to process the input prompt and generate a response using the model. The function also handles the truncation of long inputs and manages device-specific configurations (CPU or GPU).

```
def generate_response(prompt,
max_length=700):
    inputs = tokenizer(prompt,
return_tensors="pt", truncation=True,
max_length=512)
    if torch.cuda.is_available():
        inputs = {k:
v.to(model.device) for k, v in
inputs.items()}
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_length=max_length,
            temperature=0.7,
            do_sample=True,

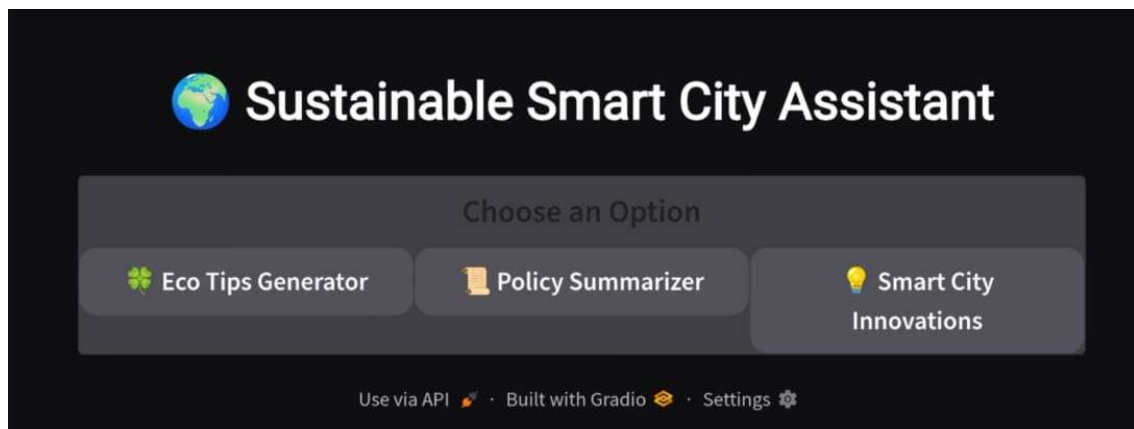
pad_token_id=tokenizer.eos_token_id
        )
    response =
tokenizer.decode(outputs[0],
skip_special_tokens=True)
    return response.replace(prompt,
"").strip()
```

Gradio UI Setup: The frontend is built using Gradio. It includes buttons for navigation between the dashboard and different pages (eco tips, policy summarizer, smart city innovations). Each page includes relevant inputs and output areas for user interaction.

```
with gr.Blocks(css="""  
body {background:  
linear-gradient(120deg,#89f7fe,#66a6ff  
);}  
#heading  
{text-align:center;font-size:36px;font  
-weight:bold;color:#fff;font-family:Ve  
rdana;margin:20px;}  
button  
{border-radius:12px;font-size:18px;font  
-weight:bold;padding:10px  
16px;transition:0.3s;}  
button:hover {transform:scale(1.05);}  
""") as app:
```

5. User Interface Design

- **Main Dashboard:** The dashboard is the starting point of the application. It presents the user with three main options:
- **Eco Tips Generator:** Button to enter the eco tips section.
- **Policy Summarizer:** Button to enter the policy summarization section.
- **Smart City Innovations:** Button to enter the smart city ideas section.
- **Eco Tips Page:** A text input field allows users to enter keywords, and a button generates eco-friendly tips based on the input.
- **Policy Summarizer Page:** Users can paste a policy document, and a button generates a summary.
- **Smart City Innovations Page:** Users can input topics related to smart cities, and a button generates innovative ideas.
- **Navigation:** Each page includes a "Back to Dashboard" button for easy navigation.



6. Future Enhancements

- **User Authentication:** Implement user login and profile management to personalize suggestions.
- **Data Persistence:** Store user-generated eco tips, summaries, and smart city ideas for future reference.
- **Multi-language Support:** Add multilingual capabilities to make the assistant accessible to a wider audience.
- **Mobile App:** Develop a mobile version of the application for better accessibility.

7. Conclusion

Summary:

The Sustainable Smart City Assistant is a comprehensive tool that aims to promote sustainability and innovation in urban planning. It leverages powerful language models to provide users with practical tips, policy summaries, and innovative ideas for building smarter, greener cities.

Potential Impact:

This tool can serve as a valuable resource for individuals, businesses, and policymakers looking to implement sustainable practices and smart city innovations.